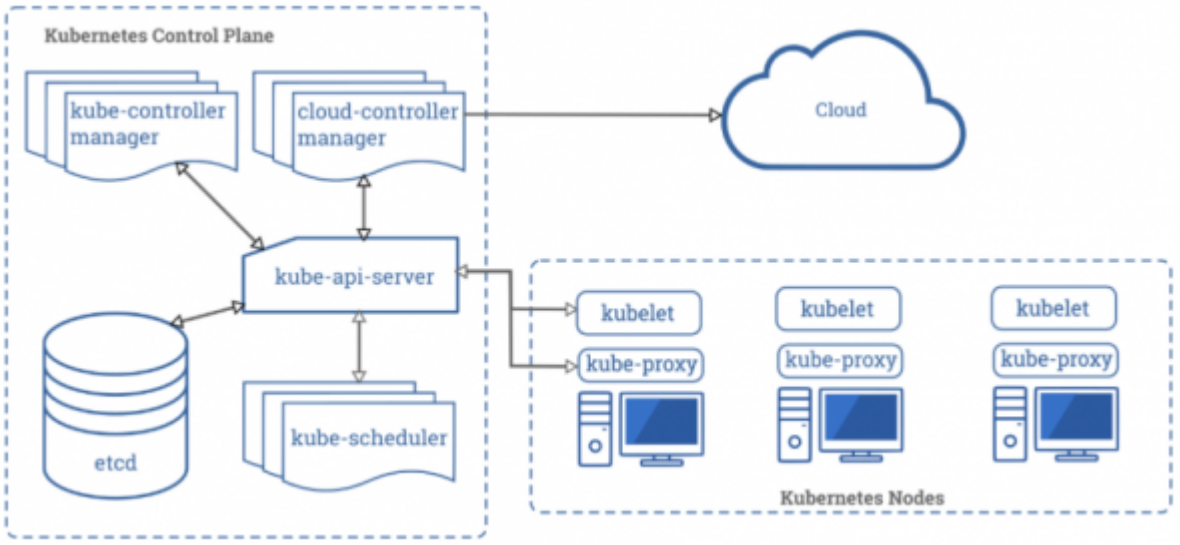
Ch4. 쿠버네티스 아키텍처

쿠버네티스 클러스터의 전체 구조

쿠버네티스 클러스트는 크게 2 종류의 서버로 구성

- 마스터 노드: 클러스터를 관리
- 워커 노드: 실제 컨테이너를 실행



- 마스터 노드
  - etcd, kube-apiserver, kube-scheduler, kube-controller-namager, kubelet, kube-proxy, docker 등의 컴포넌트가 실행
  - 보통 고가용성을 만족하고자 서버 3대 정도 구성해서 운영. 평소 실제 클러스터를 관리하는 리더 마스터는 1대이고 나머지 2대는 대기한다.
  - 리더 마스터에 장애가 발생하면 자연스럽게 나머지 2대 중 1대가 리더 역할을 맡을 수 있다.
- 워커 노드
  - kubelet, kube-proxy, docker 등의 컴포너트가 실행
  - 실제 사용하는 컨테이너 대부분은 노드에서 실행

쿠버네티스의 주요 컴포넌트

- 쿠버네티스는 근본적으로 클러스터를 관리한다. 클러스터는 원래 단일 컴퓨터가 아니라 여러 대 컴퓨터를 하나의 묶음으로 다루는 것을 의미한다.
- 쿠버네티스의 컴포넌트는 3가지로 구분
    - 마스터용 컴포넌트, 워커용 컴포넌트, 필수는 아니지만 추가로 사용할 수 있는 애드온용 컴포넌트

1. 마스터용 컴포넌트

- 마스터용 컴포넌트들은 실제 클러스터 전체를 관리

etcd

- 고가용성을 제공하는 키-값 저장소
- 쿠버네티스에서 필요한 모든 데이터를 저장하는 데이터베이스 역할

kube-apiserver

- 쿠버네티스 클러스터의 API를 사용할 수 있도록 하는 컴포넌트
- 클러스터로 온 요청이 유효한지 검증

kube-scheduler

- 현재 클러스터 안에서 자원할당이 가능한 노드 중 알맞은 노드를 선ㅌ택해서 새롭게 만든 파드를 실행

kube-controller-manager

- 쿠버네티스는 파드들을 관리하는 컨트롤러가 있다.
- 컨트롤러 각각은 논리적으로 개별 프로세스지만 복잡도를 중이려고 모든 컨트롤러를 바이너리 파일 하나로 컴파일해 단일 프로세스로 실행
- 컨트롤러 각각을 실행하는 컴포넌트

cloud-controller-manager

- 쿠버네티스의 컨트롤러들을 클라우드 서비스와 연결해 관리하는 컴포넌트
- 보통 4가지 컨트롤러 컴포넌트를 관리
    - 노드 컨트롤러
    - 라우트 컨트롤러
    - 서비스 컨트롤러

- 볼륨 컨트롤러

2. 워커노드용 컴포넌트

kubelet

- 클러스터 안 모든 노드에서 실행하는 에이전트
- 파드 컨테이너들의 실행을 직접 관리

kube-proxy

- 클러스터 안에 별도의 가상 네트워크를 설정하고 관리

컨테이너 런타임

- 실제로 컨테이너를 실행
- 가장 많이 알려진 런타임으로 docker, containerd, runc 같은 런타임이 있다.
- 보통 컨테이너 표준을 정하는 OCI(Open Container Initiative)의 런타임 규격을 구현한 컨테이너 런타임이라면 쿠버네티스에서 사용할 수 있다.

3. 애드온

Addons

- 클러스터 안에서 필요한 기능을 실행하는 파드

네트워킹 애드온

- CNI=Container Network Interface
- 클러스터 안에 가상 네트워크를 구성해 사용할 때 kube-proxy 이외에 네트워킹 애드온(네트워킹 플러그인)을 사용
- AWS, Azure, Google Cloud 플랫폼 같은 클라우드 서비스에 제공하는 쿠버네티스를 사용한다면 별도의 네트워킹 애드온을 제공하므로 신경쓰지 않아도 된다.
- 쿠버네티스를 직접 서버에 구성한다면 네트워킹 관련 애드온을 설치해서 사용해야 한다.
- 네트워킹 애드온: Flannel, Calico, Cilium, Weave Net 등

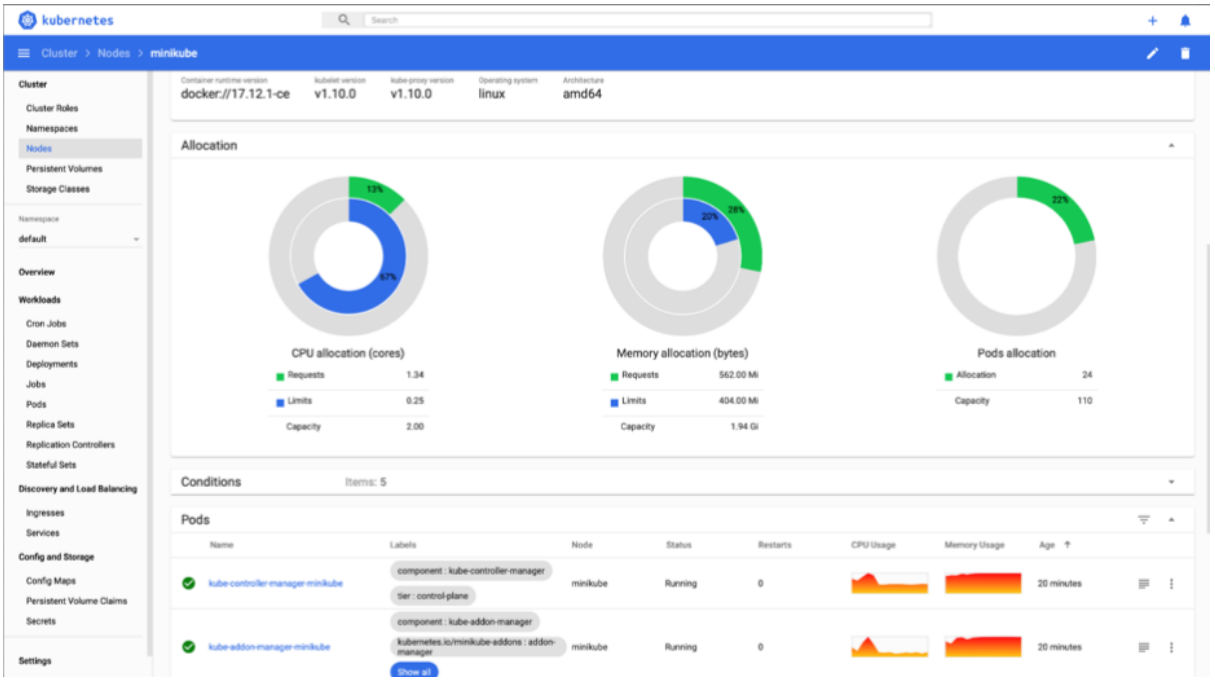
DNS 애드온

- 쿠버네티스 안에서 동작하는 DNS 서버

- 쿠버네티스 서비스에 DNS 레코드를 제공
- 쿠버네티스 안에 실행된 컨테이너들은 자동으로 DNS 서버에 등록
- 주로 사용하는 DNS 애드온: kube-dns와 CoreDNS
    - 최근에는 CoreDNS를 기본 DNS 애드온으로 사용

대시보드 애드온

- 쿠버네티스는 kubectl이라는 CLI를 많이 사용
- 하지만 웹 UI로 쿠버네티스를 사용할 필요가 있을 때 대시보드 애드온을 사용할 수 있다.



컨테이너 자원 모니터링

- 클러스터 안에서 실행 중인 컨테이너의 상태를 모니터링하는 애드온
- CPU 및 메모리 사용량 같은 데이터들을 시계열 형식으로 저장해서 볼 수 있다.
- kubelet 안에 포함된 **cAdvisor**라는 컨테이너 모니터링 도구를 사용
- cAdvisor에서는 자원 사용량 데이터를 수집하는 메트릭 서버를 손쉽게 모니터링에 이용

클러스터 로깅

- 클러스터 안 개별 컨테이너의 로그와 쿠버네티스 구성요소의 로그들을 중앙화한 로그 수집 시스템에 모아서 보는 애드온

- 로그를 수집해서 보여줄 때는 **ELK(ElasticSearch, Logstash, Kibana)**나 **EFK(ElasticSearch, Fluentd, Kibana)**를 많이 사용

오브젝트와 컨트롤러

- 쿠버네티스는 크게 오브젝트와 오브젝트를 관리하는 컨트롤러로 나뉜다.
- 사용자는 템플릿 등으로 쿠버네티스에 자원의 '바라는 상태(Desired State)를 정의하고 컨트롤러는 바라는 상태와 현재 상태가 일치하도록 오브젝트들을 생성/삭제한다.
- 오브젝트: 파드, 서비스, 볼륨, 네임스페이스 등이 있다.
- 컨트롤러: 레블리카세트, 디플로이먼트, 스테이트풀세트, 데몬세트, 잡 등이 있다.

1. 네임스페이스

- 쿠버네티스 클러스터 하나를 여러 개 논리적인 단위로 나눠서 사용하는 것
- 네임스페이스 덕분에 쿠버네티스 클러스터 하나를 여러 개 팀이나 사용자가 함께 공유할 수 있다.
- 또한 클러스터 안에서 용도에 따라 실행해야 하는 앱을 구분할 수도 있다.
- 네임스페이스별로 별도의 쿼터를 설정해서 특정 네임스페이스의 사용량을 제한할 수 있다.
- 쿠버네티스를 처음 설치하면 기본으로 몇 개의 네임스페이스가 생성된다.

kubectl get namespaces

- 현재 생성되어 있는 네임스페이스 확인

```
$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   7d21h
kube-node-lease   Active   7d21h
kube-public       Active   7d21h
kube-system       Active   7d21h
```

- default, kube-system, kube-public, kube-node-lease 등은 쿠버네티스가 기본으로 생성하는 네임스페이스, docker는 도커로 설치한 쿠버네티스이므로 도커에서 만든 네임스페이스

- default: 기본 네임스페이스. 쿠버네티스에서 명령을 실행할 때 별도의 네임스페이스를 지정하지 않는다면 항상 default 네임스페이스에 명령을 적용
- kube-system: 쿠버네티스 시스템에서 관리하는 네임스페이스. 쿠버네티스 관리용 파드나 설정이 있다.
- kube-public: 클러스터 안 모든 사용자가 읽을 수 있는 네임스페이스. 보통 클러스터 사용량 같은 정보를 이 네임스페이스에서 관리
- kube-node-lease: 각 노드 임대 오브젝트를 관리하는 네임스페이스

kubectl config current-context

- 현재 컨텍스트 정보를 확인

```
$ kubectl config current-context
kubernetes-admin@kubernetes
```

kubectl config get-contexts 컨텍스트명

- 컨텍스트의 정보를 확인

```
$ kubectl config get-contexts kubernetes-admin@kubernetes
CURRENT   NAME                          CLUSTER      AUTHINFO
*         kubernetes-admin@kubernetes   kubernetes   kubernet
```

- NAMESPACE 부분이 비어 있다. 이는 기본 네임스페이스가 default라는 뜻

kubectl config set-context 컨텍스트명 --namespace=kube-system

- 기본 네임스페이스를 kube-system으로 변경한다.

```
$ kubectl config set-context kubernetes-admin@kubernetes --na
Context "kubernetes-admin@kubernetes" modified.
```

kubectl config get-contexts 컨텍스트명

- NAMESPACE 부분이 kube-system으로 변경된 것을 확인

```
$ kubectl config get-contexts kubernetes-admin@kubernetes
CURRENT   NAME                          CLUSTER      AUTHINFO
*         kubernetes-admin@kubernetes   kubernetes   kubernet
```

kubectl config view | grep namespace

- 기본 네임스페이스가 제대로 변경됐는지 확인

```
$ kubectl config view | grep namespace
    namespace: kube-system
```

kubectl get pods --all-namespaces = kubectl get pods -A

- 각 파드의 기본 네임스페이스가 어떻게 설정했는지 확인하고자 할 때

```
$ kubectl get pods --all-namespaces
NAMESPACE     NAME                                READY   STATUS
kube-system   coredns-5dd5756b68-2k8xt            1/1     Runnin
kube-system   coredns-5dd5756b68-bbmhx            1/1     Runnin
kube-system   etcd-master                         1/1     Runnin
kube-system   kube-apiserver-master               1/1     Runnin
kube-system   kube-controller-manager-master      1/1     Runnin
kube-system   kube-proxy-cbvvm                    1/1     Runnin
kube-system   kube-proxy-kdv4x                    1/1     Runnin
kube-system   kube-proxy-pqfz4                    1/1     Runnin
kube-system   kube-scheduler-master               1/1     Runnin
kube-system   weave-net-ct8kf                     2/2     Runnin
kube-system   weave-net-hnsmp                     2/2     Runnin
kube-system   weave-net-k5x27                     2/2     Runnin
```

kubectl config set-context 컨텍스트명 --namespace=default

kubectl config set-context 컨텍스트명 --namespace=""

- 기본 네임스페이스를 다시 default로 바꿀 때

템플릿

- 쿠버네티스 클러스터의 오브젝트나 컨트롤러가 어떤 상태여야 하는지를 적용할 때 YAML 형식의 템플릿을 사용
- YAML은 JSON과 비교했을 때 간결하며, 주석도 지원하므로 가독성이 좋다. 단, 들여쓰기에 따라 구조가 바뀌므로 작성할 때 올바르게 들여쓰기했는지 반드시 확인해야 한다.
- YAML은 'Scalars(strings/numbers)', 'Sequences(arrays/lists)', 'Mappings(hashes/dictionaries)'라는 3가지 기초요소로 표현

- 주석은 '#'으로 시작
- 여러 줄 주석은 지원하지 않는다.
- '---'은 성격이 다른 YAML 형식의 문서 여러 개가 있을 때 구분자로 사용. YAML의 시작을 알리는 용도로도 사용

템플릿의 기본 형식

```
---
apiVerstion:
kind:
metadata:
spec:
```

- apiVersion: 사용하려면 쿠버네티스 API 버전을 명시. 쿠버네티스는 버전 변경이 빠른 편. 정확하게 지정하는 것이 중요. **kubectl api-versions** 명령으로 현재 클러스터에서 사용 가능한 API 버전을 확인할 수 있다.
- kind: 어떤 종류의 오브젝트 혹은 컨트롤러의 작업인지 명시
- metadata: 메타데이터를 설정. 해당 오브젝트의 이름이나 레이블 등을 설정
- spec: kind가 파드인 경우 어떤 컨테이너를 갖고 실행하며, 실행할 때 어떻게 동작해야 할지 명시

kubectl explain 타입

- 어떤 필드가 있고 어떤 역할을 하는지를 확인

```
$ kubectl explain pods
KIND:       Pod
VERSION:    v1

DESCRIPTION:
```

```
    Pod is a collection of containers that can run on a host.
    created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
    APIVersion defines the versioned schema of this represent
    Servers should convert recognized schemas to the latest i
    may reject unrecognized values. More info:
    https://git.k8s.io/community/contributors/devel/sig-archi

  kind  <string>
...
```

- 필드 설명 없이 특정 필드와 그 아래에 속한 모든 하위 필드를 한꺼번에 보려면 --recursive 옵션을 사용

```
$ kubectl explain pods --recursive
KIND:       Pod
VERSION:    v1

DESCRIPTION:
    Pod is a collection of containers that can run on a host.
    created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind  <string>
  metadata  <ObjectMeta>
    annotations <map[string]string>
    creationTimestamp   <string>
...
```